I302 - Aprendizaje Automático y Aprendizaje Profundo

Trabajo Práctico 3: Red Neuronal

Matías Gómez Taddeo

13 de mayo de 2025

Ingeniería en Inteligencia Artificial

Resumen

Se implementaron y evaluaron múltiples modelos de redes neuronales para una tarea de clasificación multiclase, utilizando tanto una implementación propia como la librería PyTorch. Se exploraron diversas configuraciones arquitectónicas y combinaciones de hiperparámetros, aplicando criterios de validación para seleccionar la mejor configuración. El modelo que obtuvo el mejor desempeño fue M1, correspondiente a la implementación propia con una arquitectura seleccionada mediante experimentación de combinaciones, alcanzando el mayor nivel de precisión en el conjunto de validación, con un *accuracy* de 0.70 en el conjunto de validación.

1. Introducción

El presente trabajo se propuso desarrollar y analizar modelos de redes neuronales para una tarea de clasificación multiclase sobre un conjunto de imágenes en escala de grises de 28 $\times$ 28 píxeles que representan caracteres japoneses, distribuidos en 49 clases posibles.

Inicialmente, se implementó desde cero una red neuronal completamente conectada, para luego ir progresivamente incluyendo mecanismos como regularización L2, *dropout*, *learning rate scheduling*, *early stopping* y algoritmos de optimización como SGD y Adam. Posteriormente, se exploraron distintas combinaciones de hiperparámetros y arquitecturas, utilizando técnicas de validación temprana para seleccionar las configuraciones más prometedoras. También se replicaron estas experimentaciones en PyTorch, con el objetivo de comparar y analizar diferencias entre ambas implementaciones.

Finalmente, se seleccionó como modelo más efectivo a M1, correspondiente a la implementación propia con los mejores hiperparámetros identificados mediante una búsqueda de combinaciones y una arquitectura óptima definida a partir de la exploración realizada en PyTorch.

2. Métodos

2.1. Procesamiento de datos:

Como primera instancia, se realizó una partición del dataset en tres subconjuntos: entrenamiento, validación y test. Se utilizó un 80 % de los datos para conformar el conjunto de desarrollo (compuesto por un 80 % para entrenamiento y un 20 % para validación), y el 20 % restante se destinó al conjunto de test. Las imágenes fueron normalizadas dividiendo sus valores de píxeles por 255, de forma tal que sus intensidades quedaran en el rango $[0, 1]$. Además, las etiquetas fueron transformadas a formato *one-hot encoding* (OHE) para poder usarlas en la función de pérdida.

2.2. Implementación y entrenamiento de una red neuronal básica (modelo M0)

2.2.1. Implementación:

Se implementó una red neuronal completamente conectada (modelo M0), que sirvió como base para las posteriores modificaciones incorporadas en el modelo M1. Para este modelo se desarrolló una clase en Python, parametrizable en cuanto a la cantidad de capas y neuronas

por capa. Las funciones de activación utilizadas fueron *ReLU* para las capas ocultas y *Softmax* para la capa de salida.

Se programaron los algoritmos correspondientes a la pasada hacia adelante (*forward pass*) y a la retropropagación (*backward pass* o *backpropagation*). Como función de costo se utilizó la entropía cruzada, la cual permite cuantificar la diferencia entre la probabilidad predicha (resultado de la capa *Softmax*) y el vector objetivo en formato *one-hot encoding*.

Durante ambas pasadas se actualizaron diccionarios internos que representan el estado del modelo en cada época. Estos almacenan las activaciones, los logits, los gradientes, los deltas (errores por capa), y los pesos. Con esta información, y en base a una tasa de aprendizaje definida, se implementó el algoritmo de actualización de pesos que es en definitiva el aprendizaje del modelo a lo largo de las épocas.

2.2.2. Entrenamiento y métricas:

Este modelo básico fue entrenado con una arquitectura de 2 capas ocultas de 100 y 80 nodos respectivamente, con una tasa de aprendizaje de 0.1 y durante 400 épocas. La evolución del costo se observa en el siguiente gráfico:

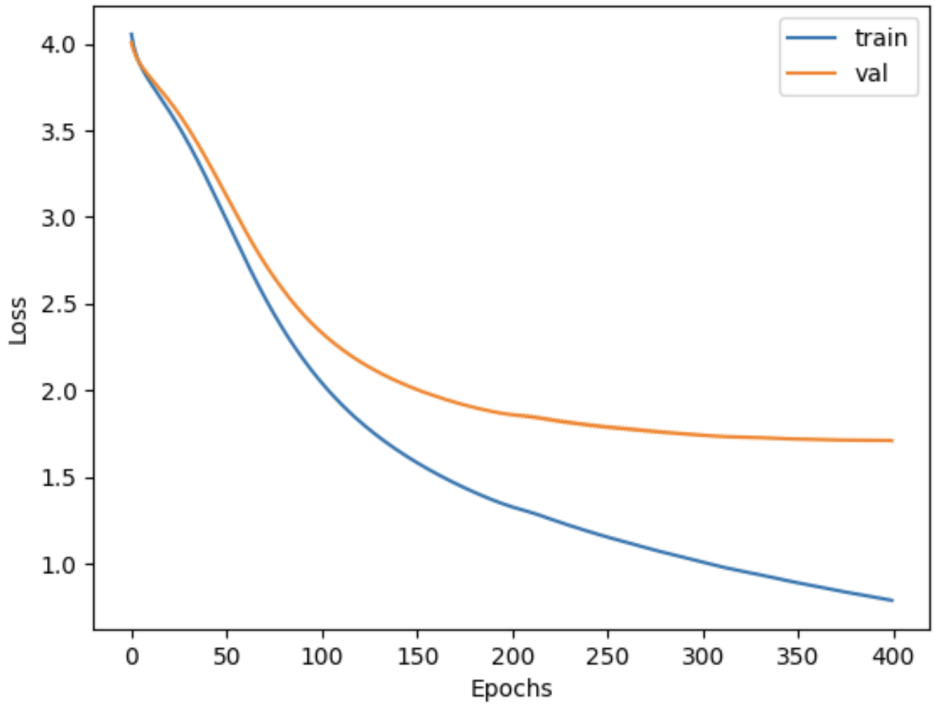

Figura 1: Curva de evolución de costo de modelo M0 en conjuntos de entrenamiento y de validación.

Se puede observar en el gráfico 1 que, mientras la función de costo en entrenamiento continúa disminuyendo, alrededor de la época 400 la curva correspondiente a validación alcanza una meseta. Comprobaciones adicionales con mayor cantidad de épocas confirmaron que, a partir de ese punto, ambas curvas comienzan a divergir.

Además, se calculó la matriz de confusión tanto para el conjunto de entrenamiento como para el de validación. En la figura siguiente se presenta esta última:

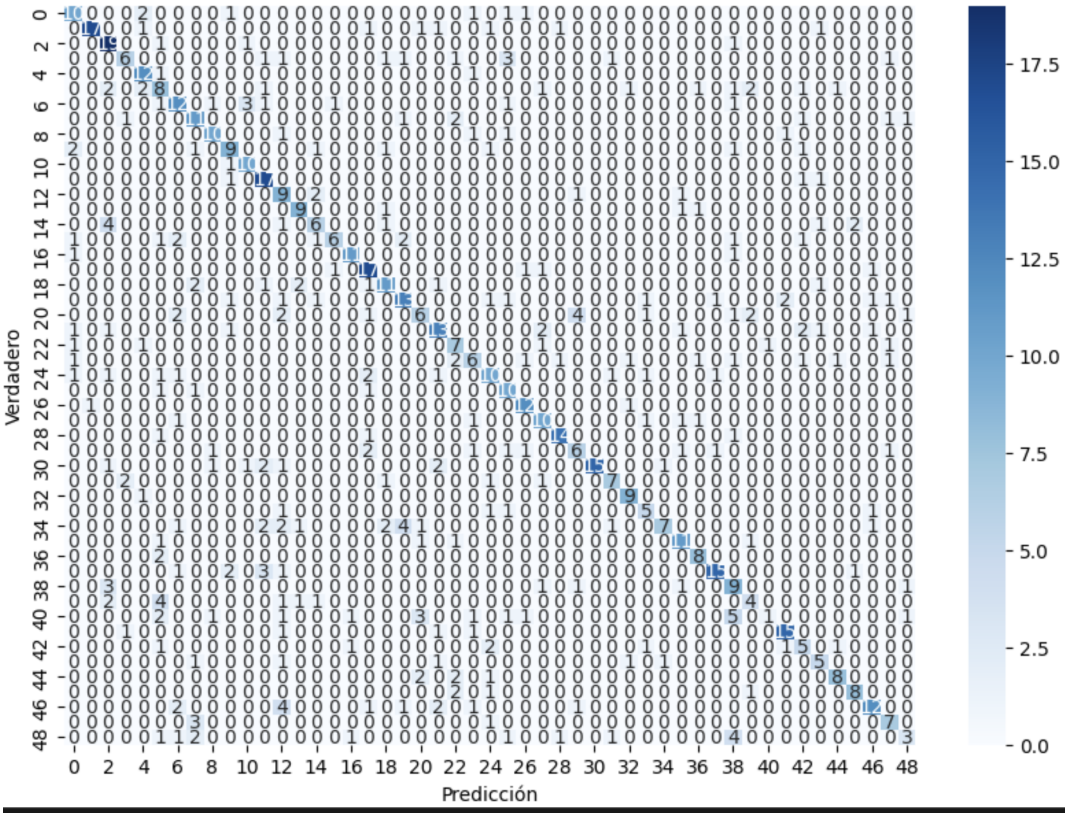

Figura 2: Matriz de confusión del modelo M0 sobre el conjunto de validación.

En la figura 2 se observa que la mayor densidad de valores se encuentra sobre la diagonal, lo que indica que la mayoría de las muestras fueron correctamente clasificadas. El desempeño global del modelo, medido mediante *accuracy*, fue de 0,81 en el conjunto de entrenamiento y 0,59 en el de validación.

2.3. Implementación y entrenamiento de una red neuronal avanzada (modelo M1):

2.3.1. Implementación:

Sobre la base del modelo M0, se implementaron las siguientes mejoras:

1. *Rate scheduling* lineal y exponencial.
2. Descenso por gradiente estocástico (SGD) con mini-batch.
3. Optimizador ADAM.
4. Regularización L2.

5. Regularización mediante *early stopping*.

6. Regularización mediante *dropout*.

Se tomaron decisiones en cuanto a la forma de implementarlo. Por ejemplo, en el *rate scheduling* se optó por la opción de pasar por parámetro una tasa de aprendizaje inicial y otra final, de manera que la evolución entre ambas sea lineal. En cuanto a SGD, el principal cambio es que los pesos se actualizan por lote, no por época como se hizo para M0. Sin embargo, la pérdida, o costo, se sigue calculando por época, para sumarlo al historial de pérdidas.

2.3.2. Entrenamiento y métricas con cada agregado:

Estas modificaciones fueron agregadas y evvaluadas de a una para su comparación, con el modelo básico y entre ellas. La arquitectura y tasa de aprendizaje utilizadas son las mismas que con M0, no así las épocas ya que cada modelo llega a su punto óptimo en distinta cantidad de épocas. Los resultados fueron los siguientes:

Modelo / Técnica	**Acc. train**	**Acc. val**	**Hiperparámetros clave**
M0 (baseline)	0.81	0.59	$lr = 0{,}1$
Rate scheduling lineal	0.68	0.54	$lr_0 = 0{,}10,\ lr_{\text{final}} = 0{,}005$
Rate scheduling exponencial	0.53	0.42	$lr_0 = 0{,}10,\ \lambda = 0{,}01$
Mini-batch SGD	0.82	**0.60**	tamaño de lote = 64
Optimizador Adam	0.56	0.43	$\beta_1 = 0{,}9,\ \beta_2 = 0{,}999,\ \varepsilon = 10^{-8}$
Regularización L2	0.81	0.57	$\lambda_{\text{L2}} = 0{,}001$
Early stopping	0.82	0.57	*patience* = 10
Dropout	0.82	0.56	$p = 0{,}5$

Cuadro 1: Resultados comparativos de las técnicas exploradas

En la tabla 1 se observan los resultados obtenidos, y se ve que por poco, el agregado que más diferencia aportó al *accuracy* en validación fue SGD.

2.3.3. Selección de la mejor combinación

Se exploraron un total de 48 combinaciones entre distintas arquitecturas y valores de hiperparámetros, con el objetivo de identificar aquella que ofreciera el mejor desempeño sobre el conjunto de validación. Algunos hiperparámetros, como la *patience* del *early stopping* y la probabilidad de *dropout*, fueron fijados a valores convenientes para todas las pruebas, con el fin de reducir el espacio de búsqueda y mantener la coherencia experimental.

Los resultados completos de esta exploración se presentan en la Tabla 3 del Apéndice A.

La combinación que obtuvo el mejor desempeño correspondió a una red con arquitectura `[512, 256]` y los siguientes hiperparámetros:

- **Optimizador:** Adam

- **Tasa de aprendizaje:** $lr = 0{,}001$
- **Regularización L2:** $\lambda = 0{,}001$
- **Dropout:** desactivado
- **Tamaño del mini-batch:** 64
- **Early stopping patience:** 40

Con esta configuración, el modelo alcanzó una **accuracy de entrenamiento de 1.00** y una **accuracy de validación de 0.70**. Se pueden observar en la figura 3 las particulares curvas de evolución de función de costo durante entrenamiento de este modelo:

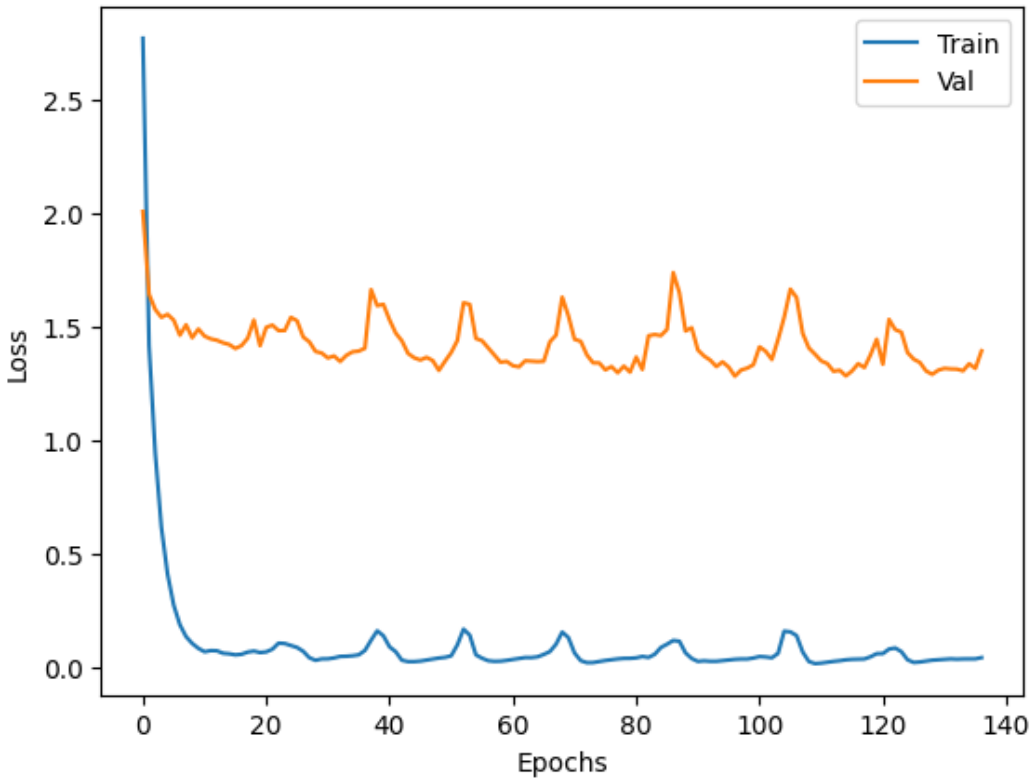

Figura 3: Curva de evolución de costo de modelo M1 en conjuntos de entrenamiento y de validación.

2.4. Desarrollo de una red neuronal con PyTorch (modelo M2):

2.4.1. Implementación del modelo M2 en PyTorch:

El modelo M2 fue implementado utilizando la biblioteca PyTorch, replicando exactamente la arquitectura y los hiperparámetros seleccionados para el modelo M1. La red se definió como una secuencia de capas lineales con activación *ReLU* intermedia y *Softmax* final implícita a través de la función de pérdida. Para el entrenamiento, se utilizaron tensores y *DataLoaders* de

PyTorch para manejar los mini-batches. Se usó como función de pérdida la entropía cruzada (nn.CrossEntropyLoss) y el optimizador Adam, incluyendo regularización L2. El entrenamiento se realizó con mecanismo de *early stopping*, restaurando los pesos correspondientes a la menor pérdida de validación. El modelo M2 permite así comparar de forma controlada el rendimiento de la implementación propia con la de PyTorch. Cabe mencionar que el modelo fue configurado para utilizar directamente la GPU del dispositivo, usando el backend MPS de PyTorch, lo que permitió acelerar el entrenamiento.

Los resultados se pueden observar en dos dimensiones: precisión y tiempo. En cuanto a lo primero, el modelo obtuvo una *accuracy* de validación de 0,64, inferior al valor alcanzado en la implementación propia con mismos hiperparámetros y arquitectura. Sin embargo, la diferencia más significativa fue el tiempo de entrenamiento: mientras en la versión implementada desde cero había demorado 19 minutos en 137 épocas, la versión de PyTorch llegó a su mejor desempeño en 46 épocas, con un tiempo de 7,6 segundos. Este es un resultado que demuestra la eficiencia de la librería, posiblemente potenciada por el uso directo de GPU.

2.4.2. Exploración de arquitecturas (modelo M3)

Para definir el modelo M3, se exploraron distintas arquitecturas variando la cantidad de capas ocultas y el número de neuronas por capa. Se evaluaron diez configuraciones diferentes, incluyendo arquitecturas más profundas y anchas, con el objetivo de encontrar la que mejor generalizara sobre el conjunto de validación. Cada red fue entrenada utilizando PyTorch, con los mismos hiperparámetros fijos empleados en el modelo M2, con el objetivo de analizar exclusivamente la influencia de la arquitectura.

De manera consistente, la arquitectura que obtuvo el mejor desempeño fue la misma utilizada en M2, compuesta por dos capas ocultas con 512 y 256 neuronas respectivamente. Esta coincidencia sugiere que la arquitectura seleccionada inicialmente para M1 y M2 resulta completa y suficiente para el problema, y que arquitecturas más complejas no aportaron mejoras en la precisión. Por lo tanto, M3 y M2 comparten tanto los hiperparámetros como la arquitectura.

2.4.3. Implementación de modelo con sobreajuste (modelo M4):

Se implementó un modelo en PyTorch que resulte en un sobreajuste sobre los datos de entrenamiento. Para implementar este modelo se realizaron varias pruebas, y se eliminaron los mecanismos de regularización que justamente buscan evitar el sobreajuste. Por esta razón, a partir del modelo M3, se eliminó el *dropout*, *early stopping* y regularización L2. A su vez, se encontró que teniendo pocas capas y pocas neuronas, pero muchas épocas durante el entrenamiento, tendía a decrecer la *accuracy* sobre el conjunto de validación. Esto se debe a que las pocas neuronas (y pesos) se ajustan -y muchas veces- sobre el set de entrenamiento, sin poder .ªlmacenarïnformación respecto al conjunto de validación. De esta manera el modelo, de los probados, que más sobreajustó se describe así:

- Arquitectura: [40, 20]
- Épocas: 400
- Optimizador: Adam

- Tasa de aprendizaje: $lr = 0{,}01$
- Regularización L2: No se utiliza
- Dropout: No se utiliza
- Tamaño del mini-batch: 32
- Early stopping: No se utiliza

Con esta configuración, el modelo alcanzó una **accuracy de entrenamiento de 0.99** y una **accuracy de validación de 0.48**. Se observó que esto parece ser escalable, por lo que posiblemente más épocas resulte en más sobreajuste. La evolución de la función de costo se puede observar en la figura 4:

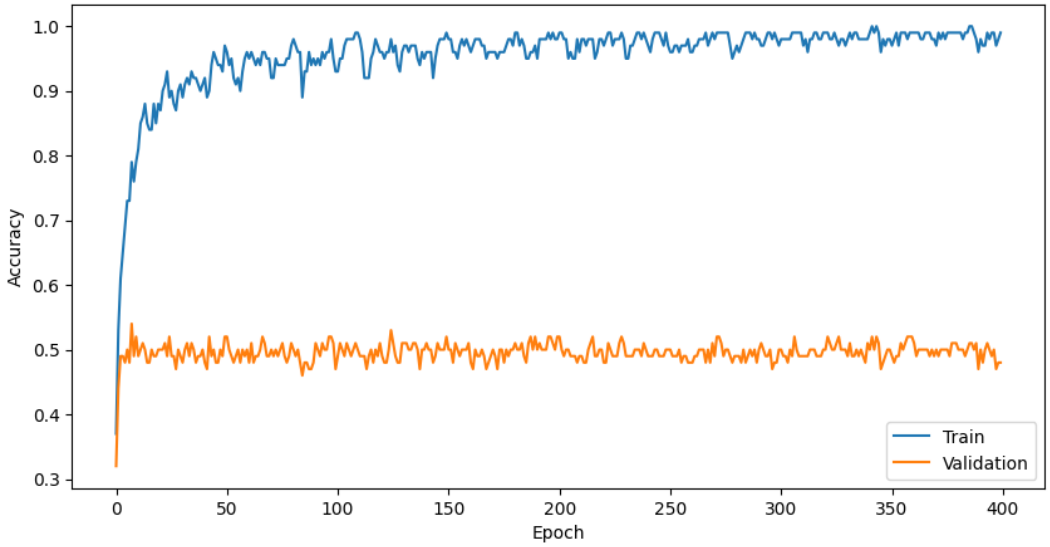

Figura 4: Curva de evolución de costo de modelo con sobreajuste (M4) en conjuntos de entrenamiento y de validación.

2.4.4. Comparación de desempeño de M0, M1, M2, M3 y M4 sobre el conjunto de test:

Habiendo entrenado los 5 modelos descritos, se procedió a hacer predicciones sobre el conjunto de test, y se compararon los resultados entre ellos. Las métricas de *accuracy* sobre el conjunto de test fueron las siguientes:

Modelo	Accuracy (test)
M0	0.63
M1	**0.69**
M2	0.68
M3	0.68
M4	0.46

Cuadro 2: *Accuracy* en conjunto de test de los distintos modelos.

En el cuadro 2 se observa que el modelo de implementación propia obtiene apenas una *accuracy* por encima de los modelos con mismos hiperparámetros y arquitectura que el entrenado con PyTorch. Si bien se desconoce la causa exacta de esta diferencia, su magnitud es tan pequeña que podría atribuirse a variaciones aleatorias propias del conjunto de test. Los resultados en test son consistentes con los que se fueron observando en los entrenamientos y la performance en validación. Esto se ve en que M2 y M3 son modelos iguales en cuanto arquitectura. No obstante el entrenamiento depende de cierta aleatoriedad, como por ejemplo el barajar los datos para la elección de los mini-batch durante el entrenamiento, lo cual puede repercutir en las métricas. A su vez, el modelo con sobreajuste fue consistente con los resultados y obtuvo un desempeño pobre al enfrentar datos nuevos.

2.5. Predicción de probabilidades sobre el dataset X_COMP.npy

Dado que el modelo con mejor desempeño en validación fue el M1, se lo seleccionó para realizar predicciones sobre un conjunto de datos nuevo y no etiquetado, `X_COMP.npy`. Este dataset contiene 1000 muestras con 784 características cada una, por lo que mantiene la misma dimensionalidad que los datos usados en el entrenamiento.

El objetivo fue calcular la probabilidad de pertenencia de cada muestra a cada una de las 49 clases posibles. Para eso, se aplicó el método *forward* del modelo M1 sobre cada muestra, obteniendo una distribución de probabilidad por fila. Estas predicciones se guardaron en una lista, luego convertida en una matriz de probabilidades, y finalmente organizada en un *DataFrame* de pandas, donde cada fila representa una muestra y cada columna una clase.

El resultado fue guardado en un archivo `CSV` llamado `predicciones.csv`, utilizando un formato de punto flotante que conserva la precisión de los valores predichos.

2.5.1. Para mejorar y explorar en el futuro:

1. Ampliar la exploración implementando una función propia de *Grid Search*.
2. Explorar y analizar las funciones internas de PyTorch.
3. Hacer más análisis de la evolución de los pesos durante el entrenamiento.

A. Resultados adicionales

Cuadro 3: Combinaciones de arquitecturas e hiperparámetros evaluadas

neurons_per_layer	optimizer	lr_0	L2	dropout	batch	val_acc	tr_acc
[512, 256]	**adam**	**0.001**	**0.001**	**False**	**64**	**0.70**	**1.00**
[256, 128, 64]	adam	0.001	0.001	False	64	0.68	1.00
[256, 128, 64]	adam	0.001	0.001	False	32	0.65	1.00
[256, 128, 64]	adam	0.0005	0.001	False	64	0.65	0.97
[128, 64]	adam	0.001	0.001	False	64	0.64	0.93
[256, 256, 128, 64]	adam	0.0005	0.001	False	32	0.63	0.94
[128, 64]	adam	0.0005	0.001	False	32	0.63	0.93
[256, 256, 128, 64]	adam	0.001	0.001	False	64	0.62	0.92
[256, 128, 64]	adam	0.0005	0.001	False	32	0.62	0.92
[128, 64]	sgd	0.050	0.000	True	32	0.61	0.90
[256, 256, 128, 64]	adam	0.0005	0.001	False	64	0.61	0.94
[128, 64]	sgd	0.020	0.001	False	32	0.61	0.88
[256, 256, 128, 64]	adam	0.001	0.001	False	32	0.61	0.94
[256, 128, 64]	adam	0.0005	0.000	True	32	0.61	0.98
[256, 128, 64]	sgd	0.020	0.001	False	32	0.61	0.93
[256, 128, 64]	sgd	0.050	0.001	True	32	0.60	0.92
[128, 64]	adam	0.0005	0.000	True	64	0.60	0.88
[128, 64]	sgd	0.050	0.000	True	64	0.60	0.89
[128, 64]	sgd	0.020	0.001	False	64	0.60	0.91
[128, 64]	sgd	0.020	0.000	False	64	0.60	0.92
[128, 64]	adam	0.001	0.001	False	32	0.60	0.82
[128, 64]	sgd	0.0200	0.000	True	64	0.60	0.92
[128, 64]	adam	0.0010	0.001	False	32	0.60	0.87
[256, 128, 64]	sgd	0.0500	0.001	False	32	0.59	0.81
[256, 128, 64]	sgd	0.0500	0.001	False	64	0.59	0.92
[128, 64]	adam	0.0005	0.000	True	32	0.59	0.89
[256, 128, 64]	sgd	0.0200	0.001	False	64	0.59	0.93
[128, 64]	sgd	0.0200	0.000	True	32	0.59	0.90
[128, 64]	sgd	0.0500	0.001	False	64	0.59	0.87
[256, 256, 128, 64]	sgd	0.0200	0.001	False	64	0.59	0.90
[128, 64]	adam	0.0010	0.000	True	32	0.59	0.88
[256, 128, 64]	adam	0.0005	0.000	True	64	0.59	0.87
[256, 256, 128, 64]	sgd	0.0500	0.001	False	64	0.58	0.86
[256, 256, 128, 64]	sgd	0.0500	0.001	False	32	0.58	0.86
[128, 64]	adam	0.0010	0.000	True	64	0.58	0.87
[256, 128, 64]	adam	0.0010	0.000	True	64	0.58	0.82
[256, 128, 64]	adam	0.0010	0.000	True	32	0.58	0.83
[256, 128, 64]	sgd	0.0200	0.000	True	64	0.58	0.89
[256, 128, 64]	sgd	0.0200	0.000	True	32	0.58	0.90
[256, 128, 64]	sgd	0.0500	0.000	True	64	0.58	0.91
[256, 256, 128, 64]	sgd	0.0200	0.001	False	32	0.57	0.88
[256, 256, 128, 64]	adam	0.0010	0.000	True	32	0.47	0.68
[256, 256, 128, 64]	adam	0.0005	10.000	True	32	0.44	0.70
[256, 256, 128, 64]	adam	0.0005	0.000	True	64	0.43	0.66
[256, 256, 128, 64]	adam	0.0010	0.000	True	64	0.41	0.56
[256, 256, 128, 64]	sgd	0.0500	0.000	True	64	0.36	0.59
[256, 256, 128, 64]	sgd	0.0500	0.000	True	32	0.33	0.53